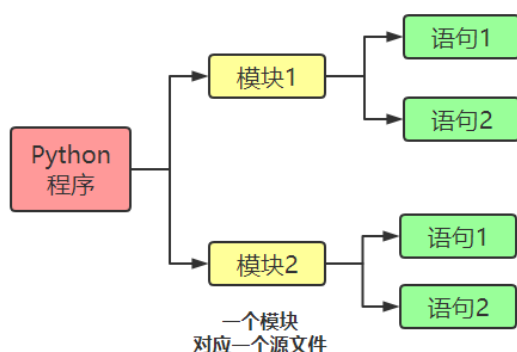


Python程序的构成



语句是程序的基本单元。
心是爱情基本单元。

- ① Python程序由模块组成。一个模块对应python源文件，一般后缀名是：.py
- ② 模块由语句组成。运行Python程序时，按照模块中语句的顺序依次执行
- ③ 语句是Python程序的构造单元，用于创建对象、变量赋值、调用函数、控制语句等

ctrl+s：随手保存写好的文件！程序员第一习惯！

代码的组织和缩进

很多编程语言通过字符(例如：花括号{})、关键字(例如：begin/end)来划分代码块。同时，在配合代码的缩进增加可读性。“龟叔”设计Python语言时，直接通过缩进来组织代码块。“缩进”成为了Python语法强制的规定。

缩进时，几个空格都是允许的，但是数目必须统一。**通常采用“四个空格”表示一个缩进。**

同时，也要避免将“tab制表符”或者tab与空格混合的缩进风格。目前，**常用的编辑器一般设置成：tab制表符就是4个空格**

Python官方推荐的PEP-8代码风格详细说明，有兴趣的同学可以参考：

<https://www.python.org/dev/peps/pep-0008/>

使用\行连接符

一程序长度是没有限制的，但是为了可读性更强，通常将一行比较长的程序分为多行。这是，我们可以使用\行连接符，把它放在行结束的地方。Python解释器仍然将它们解释为同一行。

```
1 a = [10,20,30,40,\
2     50,60,70,\
3     80,90,100]
4 b = 'abcdefg\
5     hijklmn\
6     opqrst\
7     uvwxyz'
8 print(a)
9 print(b)
```

实时效果反馈

1. 关于python代码组织，正确的说法是：

- ☐ A python中，也可以像c语言一样使用{ }表示缩进
- ☐ B python中，强制使用缩进，一般4个空格表示一个缩进
- ☐ C python中，缩进是可选的，不强制
- ☐ D python中，解释器会自动加缩进，缩进不强制写也可以

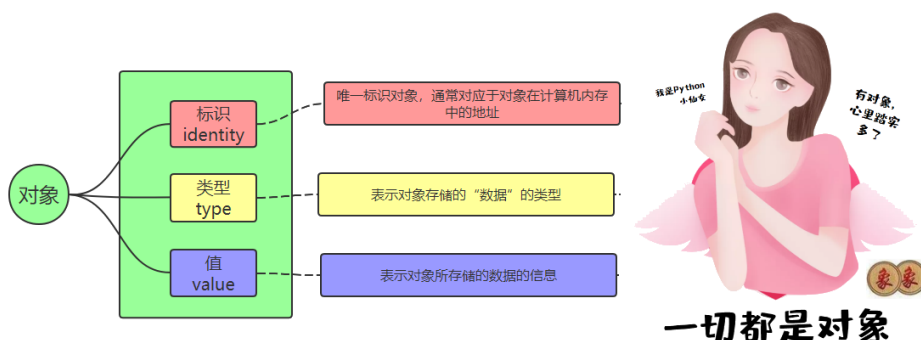
2. 一行代码较长，变成多行，使用什么符号？

- ☐ A 回车符
- ☐ B / 行连接符
- ☐ C \ 行连接符
- ☐ D 使用分号

答案

1=>B 2=>C

对象



- ① Python中，一切皆对象。
- ② 每个对象由：标识（identity）、类型（type）、value（值）组成

- ① 标识用于唯一标识对象，通常对应于对象在计算机内存中的地址。使用内置函数`id(obj)`可返回对象`obj`的标识。
- ② 类型用于表示对象存储的“数据”的类型。类型可以限制对象的取值范围以及可执行的操作。可以使用`type(obj)`获得对象的所属类型。
- ③ 值表示对象所存储的数据的信息。使用`print(obj)`可以直接打印出值。

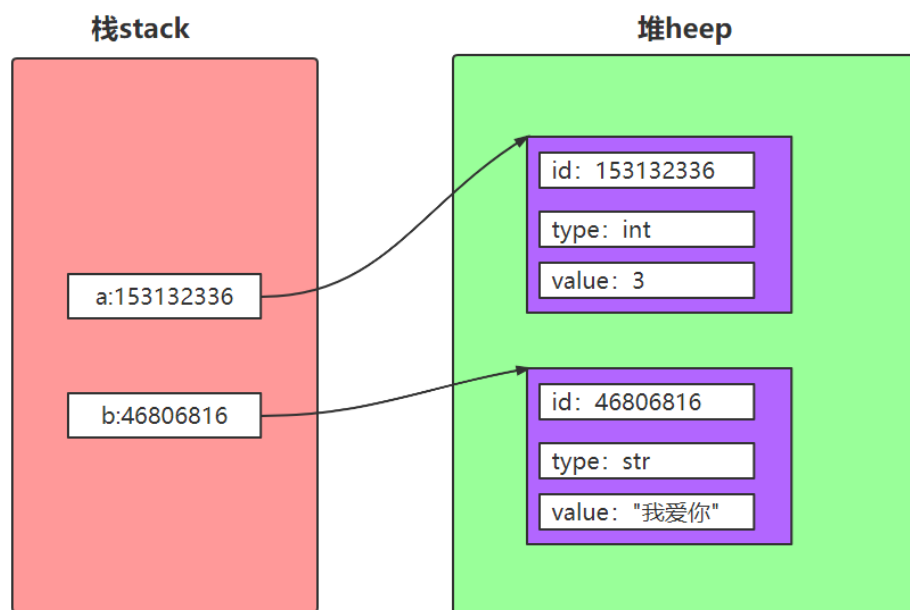
对象的本质就是：

一个内存块，拥有特定的值，支持特定类型的相关操作

源码：

```
1 a=3
2 print(a)
3 print(id(a))
4 print(type(a))
5 b='我爱你'
6 print(b)
7 print(id(b))
8 print(type(b))
```

内存示意图

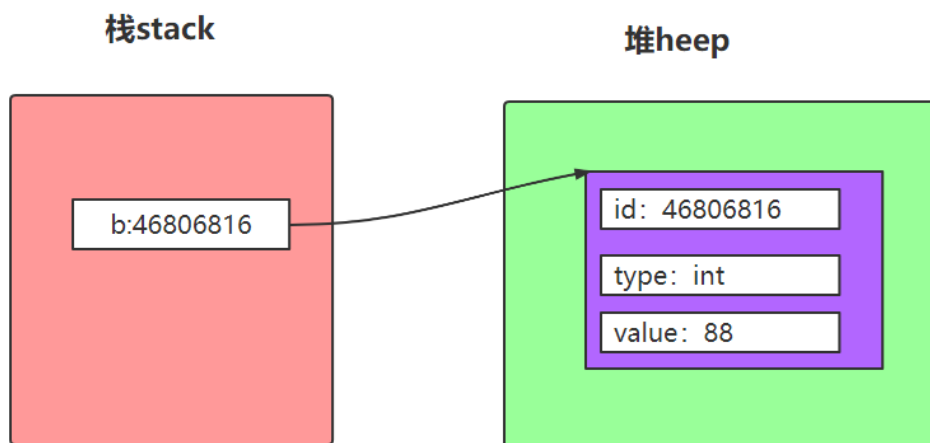


实时效果反馈

1. 对象由三部分组成，是哪三部分：

- ☒ A int, type, value
- ☐ B int, class, value
- ☐ C class, type, value
- ☐ D id, type, value

2. 如下图形，对应的代码是：



- A `b="88"`
- B `b = 46806816`
- C `b = 88`
- D `b = "int"`

答案

1=>D 2=>C

引用

在Python中，变量也称为：对象的引用(reference)。变量存储的就是对象的地址。

变量通过地址引用了“对象”。

Director of Admissions,
Massachusetts Institute of Technology,
77 Massachusetts Avenue, Cambridge,
MA 02139, USA



引用

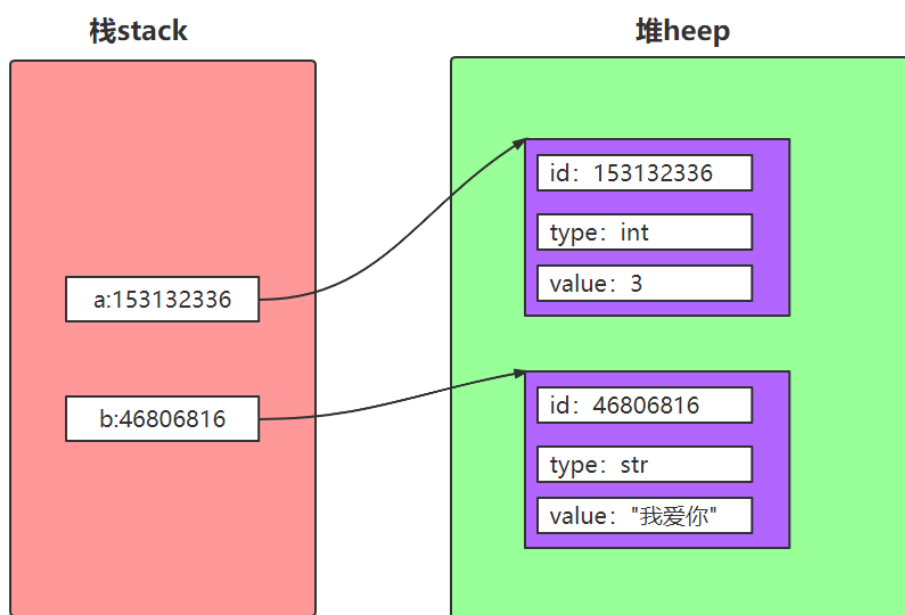
真正的对象

变量位于：栈内存（压栈出栈等细节，后续再介绍）。

对象位于：堆内存。

如下源代码对应的内存图：

```
1 a=3
2 b="我爱你"
```



⚠ Python是动态类型语言

变量不需要显式声明类型。根据变量引用的对象，Python解释器自动确定数据类型

实时效果反馈

1. 如下关于变量和对象说法，错误的是：

- A** 变量也称为引用（引用了对象的地址）
- B** 变量和对象都位于堆内存
- C** 变量没有类型，对象有类型
- D** 变量不需要声明类型，解释器可以自动确定所引用对象的类型

答案

1=>B

标识符规则



标识符4大规则

- 区分大小写
- 第一个字符必须是字母、下划线。其后的字符是：字母、数字、下划线
- 不能使用关键字。比如：`if`、`or`、`while`等
- 以双下划线开头和结尾的名称通常有特殊含义，尽量避免这种写法

基本用法

标识符规则：用于变量、函数、类、模块等的名称。标识符有如下特定的规则：

- ① 区分大小写。如：`sxt` 和 `SXT` 是不同的
- ② 第一个字符必须是字母、下划线。其后的字符是：字母、数字、下划线
- ③ 不能使用关键字。比如：`if`、`or`、`while` 等
- ④ 以双下划线开头和结尾的名称通常有特殊含义，尽量避免这种写法。比如：`__init__` 是类的构造函数

【操作】使用Python帮助系统查看关键字

```
>>> help()
```

```
help> keywords
```

```
False      def        if         raise     None      del        import     return

True       elif       in         try       and        else       is         while

as         except     lambda    with      assert    finally    nonlocal

yield

break      for        not       class     from       or         continue

global     pass
```

⚠注：无需刻意去背关键字，后面都会学习。

Python标识符命名规则

开发中，我们通常约定俗称遵守如下规则：

类型	规则	例子
模块和包名	全小写字母，尽量简单。若多个单词之间用下划线	math, os, sys
函数名	全小写字母，多个单词之间用下划线隔开	phone, my_name
类名	首字母大写，采用驼峰原则。多个单词时，每个单词第一个字母大写，其余部分小写	MyPhone、 MyClass、Phone
常量名	全大写字母，多个单词使用下划线隔开	SPEED、 MAX_SPEED

实时效果反馈

1. 如下合法的标识符是:

A `a#123`

B `100vip`

C `_good_student`

D `if`

答案

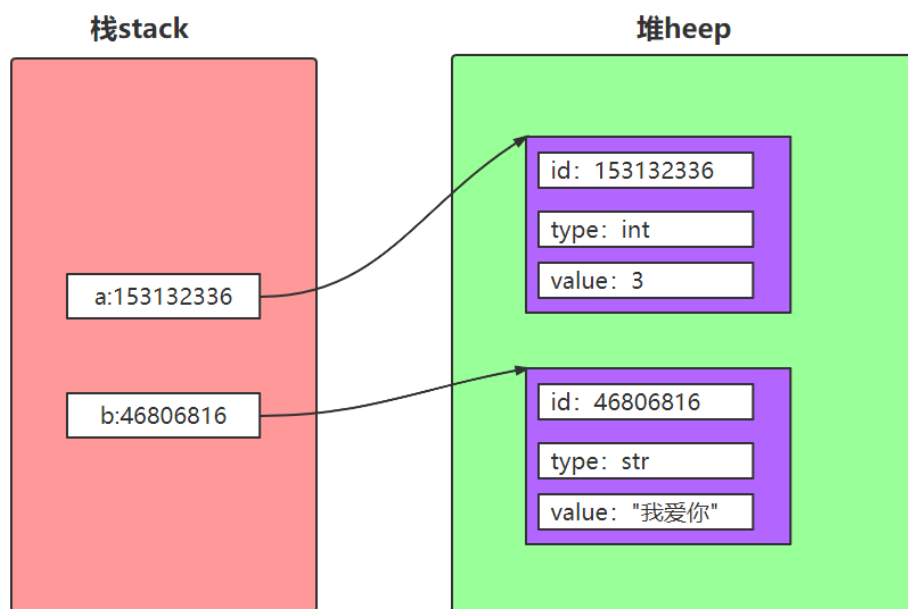
1=>C

变量和简单赋值语句

变量的声明和赋值

变量的声明和赋值：用于将一个变量绑定到一个对象上，格式：`变量`

`名 = 表达式`



最简单的表达式就是字面量。比如：`a = 123`。运行过程中，解释器先运行右边的表达式，生成一个代表表达式运算结果的对象；然后，将这个对象地址赋值给左边的变量。

【操作】变量在使用前必须先被初始化（先被赋值）

```
1 print(a)
```

报错如下：

```
1 Traceback (most recent call last):  
2   File  
  "C:\Users\webchubby\PycharmProjects\mypro01\m  
  ypy01.py", line 1, in <module>  
3     print(a)  
4 NameError: name 'a' is not defined
```

变量 `a` 在被使用前未做赋值，因此报错： `'a' is not defined`

删除变量和垃圾回收机制

- ① 可以通过del语句删除不再使用的变量。
- ② 如果对象没有变量引用，就会被垃圾回收器回收，清空内存空间。

【操作】删除变量示例

```
1 a=123  
2 del a  
3 print(a)
```

报错如下：

```
1 Traceback (most recent call last):  
2   File  
  "C:\Users\webChubby\PycharmProjects\mypro01\m  
  ypy01.py", line 3, in <module>  
3     print(a)  
4 NameError: name 'a' is not defined
```

实时效果反馈

1. 如下代码的运行，说法正确的是：

```
1 a=123  
2 del a  
3 print(a)
```

- ☐ A 可以正常运行
- ☐ B 报错。a没有被定义
- ☐ C 报错。不能删除a
- ☐ D 报错。a不能被赋值123

答案

1=>B

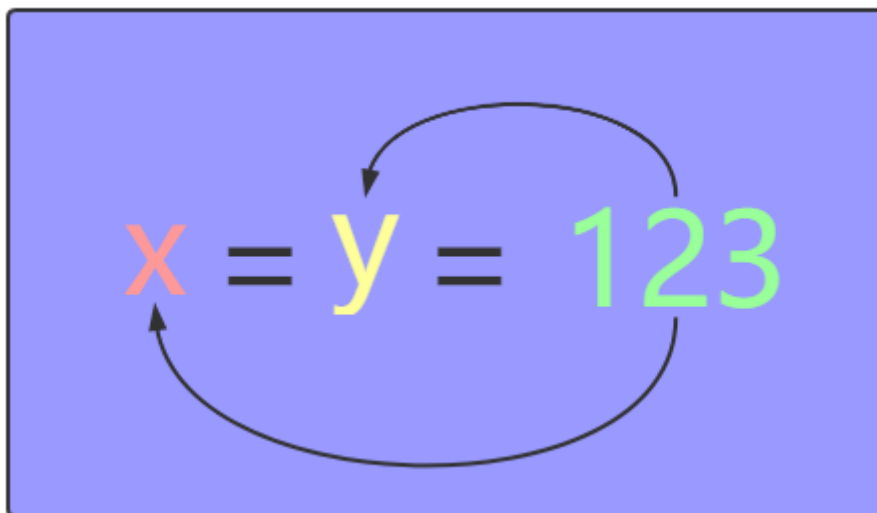
常量



Python不支持常量，即没有语法规则限制改变一个常量的值。我们只能约定常量的命名规则，以及在程序的逻辑上不对常量的值作出修改。

```
1 MAX_SPEED = 120
2 print(MAX_SPEED)    #输出120
3 MAX_SPEED = 140    # 实际是可以改的。只能逻辑上不做
   修改。
4 print(MAX_SPEED)    #输出140
```

链式赋值



链式赋值用于同一个对象赋值给多个变量。

系列解包赋值

系列数据赋值给对应相同个数的变量（个数必须保持一致）

`a,b,c=4,5,6` 相当于: `a=4;b=5;c=6`

【操作】使用系列解包赋值实现变量值交换

```
1 a,b=1,2
2 a,b=b,a #变量值互换
3 print(a,b)
```

实时效果反馈

1. 执行代码 `a,b,c=40,50,60`，说法正确的是：

- ☐ A a,b,c的值是最后的值：60
- ☐ B 只有a被赋值40。b和c是0
- ☐ C a,b,c的值是最初的值：40

D a是40, b是50, c是60

2. python中的常量, 说法正确的是:

A python中的常量是真的常量, 不能修改值

B python中的常量是逻辑上的常量, 仍然可以修改值

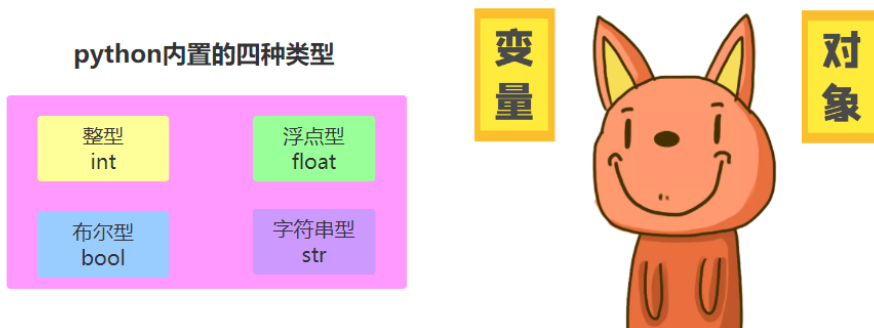
C python中的常量命名一般是小写

D python中的常量命名一般用驼峰原则, 比如:
StudentNumber

答案

1=>D 2=>B

最基本内置数据类型



变量没有类型, 但对象有类型

python中变量没有类型, 但是对象都有类型, python中最基本的内置数据类型:

① 整型 `int`

整数, `2345`, `10`, `50`

② 浮点型 `float`

小数, `3.14` 或者科学计数法 `314e-2`

③ 布尔型 `bool`

表示真假, 仅包含: `True`、`False`

4 字符串型 str

由字符组成的序列。 `"abc"`, `'sxt'`, `"尚学堂"`, `"百战程序员"`

数字和基本运算符

Python支持整数(如: `50`, `520`)和浮点数(如: `3.14`, `10.0`, `1.23e2`), 我们可以对数字做如下运算。

运算符	说明	示例	结果
+	加法	<code>3+2</code>	5
-	减法	<code>30-5</code>	25
*	乘法	<code>3*6</code>	18
/	浮点数除法	<code>8/2</code>	4.0
//	整数除法	<code>7//2</code>	3
%	模 (取余)	<code>7%4</code>	3
**	幂	<code>2**3</code>	8

【操作】基本运算符的使用

```

1 a = 7/2 #3.5
2 a = 7//2 #3
3 a = 7%2 #1
4 a = 7%4 #3
5 a = 2**3 #8
6 #a = 3/0      #报错: ZeroDivisionError: division
  by zero

```

除数为0, 会产生异常:

```

1 3/0Traceback (most recent call last): File "
  <pyshe11#31>", line 1, in <module>
  3/0ZeroDivisionError: division by zero

```

使用 `divmod()` 函数同时得到商和余数：

```
>>> divmod(13,3)
```

```
(4, 1)
```

`divmod()` 是一个函数，我们以后会详细介绍。他返回的是一个元组(后续将会学习)。

实时效果反馈

1. python中，内置的基本类型有哪些：

- ☐ A 整型、数组、布尔型、字符串型
- ☐ B 整型、浮点型、布尔型、字符型
- ☐ C 整型、浮点型、布尔型、字符串型
- ☐ D 整型、数组、布尔型、字符型

2. 如下代码，运行结果是：

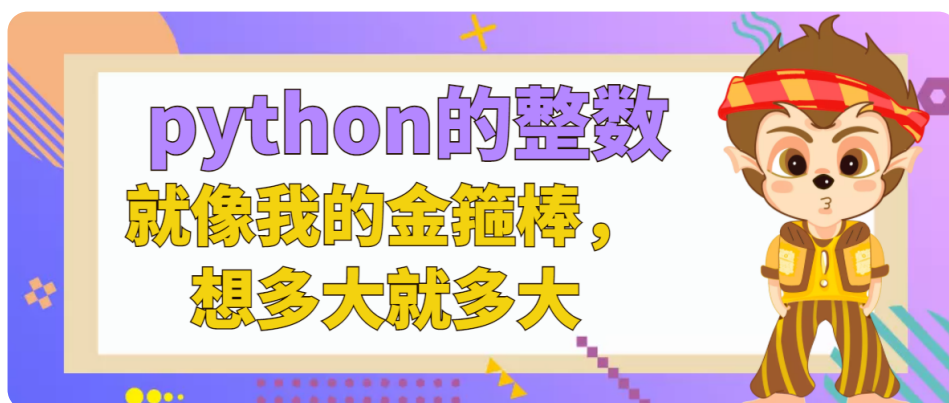
```
1 a = 7/2
2 a = 7//2
3 a = 7%2
```

- ☐ A 3, 3.5 3
- ☐ B 3.5, 3, 1
- ☐ C 3.5, 3, 3
- ☐ D 4, 3, 1

答案

1=>C 2=>B

整数



三种进制

Python中，除了十进制，还有其他三种进制：

- `0b` 或 `0B`，二进制 0 1
- `0o` 或 `0O`，八进制 0 1 2 3 4 5 6 7
- `0x` 或 `0X`，十六进制 0 1 2 3 4 5 6 7 8 9 a b c d e f

这三种进制可以非常方便的进行“位运算”操作。位运算后续将会介绍。

【操作】测试不同进制

```
1 >>> 12
2 12
3 >>> 0b101
4 5
5 >>> 0o19
6 SyntaxError: invalid syntax
7 >>> 0o10
8 8
9 >>> 0xff
10 255
```

```

11 >>> 0xf
12 15
13 >>> 0x10
14 16

```

使用int()实现类型转换:

- ① 浮点数直接舍去小数部分。如: `int(9.9)` 结果是: `9`
- ② 布尔值 `True` 转为 `1`, `False` 转为 `0`。如: `int(True)` 结果是 `1`
- ③ 字符串符合整数格式(浮点数格式不行)则直接转成对应整数, 否则报错

```

1 >>> int("456")
2 456
3 >>> int("456abc")
4 Traceback (most recent call last):
5   File "<pyshell#41>", line 1, in <module>
6     int("456abc")
7 ValueError: invalid literal for int() with
  base 10: '456abc'
8 >>> int("456.78")
9 Traceback (most recent call last):
10  File "<pyshell#42>", line 1, in <module>
11    int("456.78")
12 ValueError: invalid literal for int() with
  base 10: '456.78'
13 >>>

```

自动转型:

整数和浮点数混合运算时, 表达式结果自动转型成浮点数。比如:

`2+8.0` 的结果是 `10.0`

2. 关于python中整型和浮点型，如下说法错误的是：

- A** 16进制是0x开头
- B** `3+5.0` 结果是浮点数
- C** `int("888")` 结果是888
- D** `int("888aaa")` 结果是888

答案

1=>A 2=>D

浮点数 `float`

- ① 浮点数用科学计数法表示。比如：`3.14`，表示成：`314E-2` 或者 `314e-2`
- ② 这些数字在内存中也是按照科学计数法存储。

类型转换和四舍五入

- ① 类似于 `int()`，我们也可以使用 `float()` 将其他类型转化成浮点数。
- ② 整数和浮点数混合运算时，表达式结果自动转型成浮点数。比如：`2+8.0` 的结果是 `10.0`
- ③ `round(value)` 可以返回四舍五入的值。但不会改变原有值，而是产生新的值

增强型赋值运算符

运算符 `+`、`-`、`*`，`/`、`//`、`**` 和 `%` 和赋值符 `=` 结合可以构成“增强型赋值运算符”。

运算符	例子	等价
+=	a += 2	a = a + 2
-=	a -= 2	a = a - 2
*=	a *= 2	a = a * 2
/=	a /= 2	a = a / 2
//=	a //= 2	a = a // 2
**=	a **= 2	a = a ** 2
%=	a %= 2	a = a % 2

⚠注意:

- ① “+=”中间不能加空格!
- ② 结合的是右侧整个表达式:

1 | y *= x+2 # 相当于: y = y*(x+2) 而不是: y = y*x+2

实时效果反馈

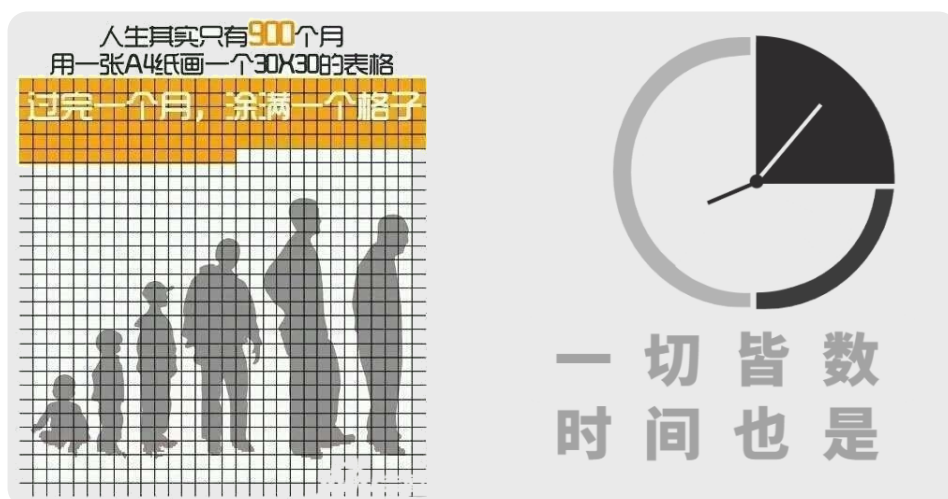
1. python3中浮点数，如下说法错误的是:

- A** 314E-2,表示的是: 3.14
- B** a+=2, 相当于a = a+2
- C** round(value) 可以进行四舍五入, 并修改了value对象的值
- D** float("3.1415")可以转成浮点数: 3.1415

答案

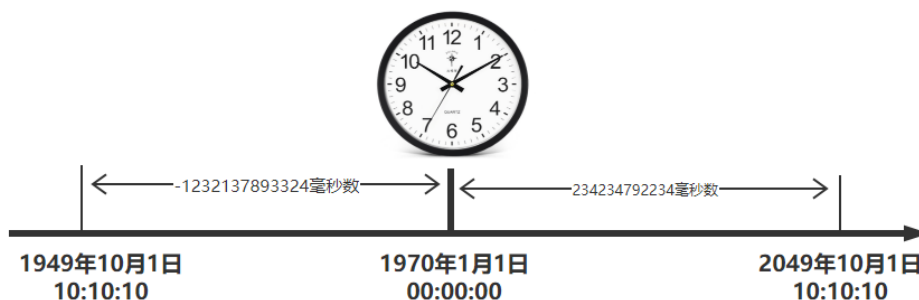
1=>C

时间的表示



计算机中时间的表示是从“1970年1月1日 00:00:00”开始，以毫秒（1/1000秒）进行计算。我们也把1970年这个时刻称为“unix时间点”。

这样，我们就把时间全部用数字来表示了。



python中可以通过`time.time()` 获得当前时刻，返回的值是以秒为单位，带微秒（1/1000毫秒）精度的浮点值。例如：

1635063628.5632517

```
1 import time
2 b = int(time.time())
3 totalMinutes = b//60
4 totalHours = totalMinutes//60
5 totalDays = totalHours//24
6 totalYears = totalDays//365 #忽略闰年情况
```

⚠️ 刚开始，我们简单认识一下时间，是为了让大家更深刻认识“一切都是数字”。关于时间格式化问题、日期问题。后面单独讲解。

实时效果反馈

1. python中，关于时间的表示，错误的是：

- A** 通过`time.time()`获得当前时刻所代表的数字，返回的是浮点数
- B** 使用`time`，需要 `import time`
- C** 时间是从1970.1.1 0:0:0 开始计算
- D** `int(time.time())`返回的是当前时刻代表的毫秒数

答案

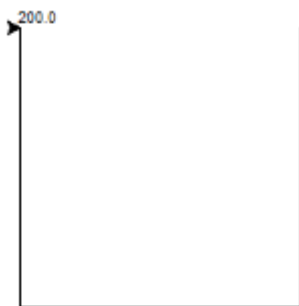
1=>D

【操作】 定义多点坐标绘出折线并计算起始点和终点距离



```
1 import turtle
2 import math
3
4 #定义多个点的坐标
5 x1,y1 = 100,100
6 x2,y2 = 100,-100
7 x3,y3 = -100,-100
8 x4,y4 = -100,100
9
10
11 #绘制折线
12 turtle.penup()
13 turtle.goto(x1,y1)
14 turtle.pendown()
15
16 turtle.goto(x2,y2)
17 turtle.goto(x3,y3)
18 turtle.goto(x4,y4)
19
20
21 #计算起始点和终点的距离
22
23 distance = math.sqrt((x1-x4)**2 + (y1-
    y4)**2)
24 turtle.write(distance)
25 turtle.done()      #该语句让程序等待，而不是直接退出窗口
```

运行结果：



实时效果反馈

1. python的海龟绘图模块的使用，错误的是：

- A** 需要导入turtle模块
- B** `turtle.done()` 可以让程序等待，而不是直接退出窗口
- C** 小海龟默认的坐标是(0,0)
- D** turtle是内置模块，不需要导入即可使用

答案

1=>D

布尔值

Python2中没有布尔值，直接用数字 `0` 表示 `False`，用数字 `1` 表示 `True`。

Python3中，把 `True` 和 `False` 定义成了关键字，但他们的本质还是 `1` 和 `0`，甚至可以和数字相加。

在Python语言底层，会将布尔值True看作1，将布尔值False看作0，尽管从表面上看，True和1、False和0是完全不同的两个值，但实际上，它们是相同的。

在Python语言中有一些特殊的布尔类型值为False，例如False、0、0.0、空值None、空序列对象（空列表、空元祖、空集合、空字典、空字符串）、空range对象、空迭代对象。其他情况，均为True。

```
1 a = True
2 b = 3
3 c = a+b      #c的值是： 4
4 print(c)
5 print('空字符串的布尔类型的值:',bool(""))
  #False
6 print('空列表布尔类型的值:',bool([]))
  #False
7 print('None布尔类型的值:',bool(None))
  #False
8 print('0布尔类型的值:',bool(0))
  #False
9 print('字符串True和False转成布尔都是
  True: ',bool("False")) #True
10
```

运算符

逻辑运算符





运算符	格式	说明
or 逻辑或	x or y	x为true, 则不计算y, 直接返回true; x为false, 则返回y
and 逻辑与	x and y	x为true, 则返回y的值x为false, 则不计算y, 直接返回false
not 逻辑非	not x	x为true, 返回false; x为false, 返回true

```

1 #测试逻辑运算符
2 a,b,c=10,20,30
3 print((a<b) and (b<c))    #and并且  输出结果是
  True
4 print((a>b) or (b>c))    #or或者  输出结果是
  False
5 print(not(b<c))          #not非  输出结果是
  False

```

比较运算符

所有比较运算符返回 `1` 表示真, 返回 `0` 表示假。这分别与特殊变量 `True` 和 `False` 等价。

以下假设变量 `a` 为 15, 变量 `b` 为 30:

运算符	描述	实例
==	等于 - 比较对象的值是否相等	(a == b) 返回 False。
!=	不等于 - 比较两个对象的值是否不相等	(a != b) 返回 true。
>	大于 - 返回x是否大于y	(a > b) 返回 False。
<	小于 - 返回x是否小于y。	(a < b) 返回 true。
>=	大于等于 - 返回x是否大于等于y。	(a >= b) 返回 False。
<=	小于等于 - 返回x是否小于等于y。	(a <= b) 返回 true。

```

1 a = 4
2 print(a<=30)
3 #关系运算符可以连用
4 if(3<a<10):
5     print("a在3和10之间")

```

位运算符

按位运算符是把数字看作二进制来进行计算的。Python中的按位运算符法则如表所示。

运算符	描述
&	按位与运算符:参与运算的两个值,如果两个相应位都为1,则该位的结果为1,否则0
	按位或运算符:只要对应的二个二进位有一个为1时,结果就为1。
^	按位异或运算符:当两对应的二进位相异时,结果为1
~	按位取反运算符:对数据的每个二进制取反,既把1变为0,把0变为1,~x类似于-x-1
<<	左移动运算符:运算数的每个二进位全部左移若干位,由"<<"右边的数据指定移动的位数,高位丢弃,低位补0
>>	右移动运算符:把">>"左边运算数的每个二进位全部右移若干位,">>"右边的数指定移动的位数

```

1 a = 0b11001
2 b = 0b01000
3
4 print(bin(a|b))    #bin()可以将数字转成二进制表示
                        '0b11001'
5 print(bin(a&b))    #与
6 print(bin(a^b))    #异或
7
8 print(3<<2)        #左移1位相当于乘以2.左移两位相当
                        于: 3*4
9 print(20>>1)       #右移移位相当于除以2

```

加法操作补充

- ① 数字相加 `3+2` 结果是 `5`
- ② 字符串拼接 `"3" + "2"` 结果是 `"32"`
- ③ 列表、元组等合并 `[10,20,30]+[5,10,100]` 结果是 `[10,20,30,5,10,100]`

乘法操作补充

- ① 数字相乘 `3*2` 结果是 `6`
- ② 字符串复制 `"sxt" *3` 结果是 `"sxtsxtsxt"`
- ③ 列表、元组等复制 `[10,20,30]*3` 结果是 `[10,20,30,10,20,30,10,20,30]`

增强赋值运算符(补充)

复合赋值可以让程序更加精炼，提高效率。

运算符	描述	示例	等价于
+=	加法赋值字符串拼接	sum += na += "sxt"	sum = sum + na = a + "sxt"
-=	减法赋值	num1 -= n	num = num - n
*=	乘法赋值	a *= b	a = a * b
/=	浮点除赋值	a/=b	a = a / b
//=	整数除赋值	a//=b	a = a//b
%=	取余赋值	a%=b	a = a % b
=	幂运算赋值	a=2	a = a**2
<<=	左移赋值	a<<=2	a = a<<2
>>=	右移赋值	a>>=2	a = a>>2
&=	按位与赋值	a&=b	a = a&b
=	按位或赋值	a =b	a=a b
^=	按位异或赋值	a^=b	a = a^b

⚠️与 C 和 JAVA 不一样, Python 不支持自增(++)和自减(--)

实时效果反馈

1. python中关于布尔类型说明, 错误的是:

- A `bool("")` 结果是: `False`
- B `bool(0)` 结果是: `False`
- C `bool("False")` 结果是: `False`
- D `c=True+5` 结果是: `6`

2. 如下代码, 正确结果是:

```

1 a,b,c=10,20,30
2 print((a<b) and (b<c))
3 print((a>b) or (b>c))
4 print(not(b<c))

```

A False False False

B False True False

C False False True

D True False False

答案

1=>C 2=>D

同一运算符



同一运算符用于比较两个对象的存储单元，实际比较的是对象的地址。

运算符	描述
is	is是判断两个标识符是不是引用同一个对象
is not	is not 是判断两个标识符是不是引用不同对象

is 与 == 区别：

is 用于判断两个变量引用对象是否为同一个，既比较对象的地址。

`==` 用于判断引用变量引用对象的值是否相等，默认调用对象的 `__eq__()` 方法。

```

1 a=20
2 b=20
3 c=30
4 print("a和b是同一个对象",a is b)    #执行结果:True
5 print("a和c是同一个对象",a is c)    #执行结果False
6 print("a和c不是同一个对象",a is not c)    #执行结果True

```

整数缓存问题

- ① 命令行模式下，Python仅仅对比较小的整数对象进行缓存（范围为[-5, 256]）缓存起来 C语言底层用数组实现，连续分配空间，便于查找，而并非是所有整数对象。
- ② 文件模式下，所有数字都会被缓存，范围是：[-无穷大,+无穷大]
 - ① 缓存实现：[-5,256]仍然 底层用数组实现；不在[-5,256]出现的数，缓存到 链表中，不连续分配空间。

总结

- ① `is` 比较两个对象的 `id` 值是否相等，是否指向同一个内存地址
- ② `==` 比较的是两个对象的内容是否相等，值是否相等
- ③ `is` 运算符比 `==` 效率高，在变量和 `None` 进行比较时，应该使用 `is`

【操作】同一运算符测试


```

1 a = 1000
2 b = 1000
3 a == b    #True
4 a is b    #命令行下是False。 文件下执行是True
5 c = 10
6 d = 10
7 c is d    #True

```

成员运算符

成员运算符测试实例中包含了一系列的成员，包括字符串，列表或元组。

运算符	描述
in	如果在指定的序列中找到值返回 True，否则返回 False
not in	不在，则返回True

```

1 a = "python"
2 b = "py"
3 print(b in a)          #True
4 c = [10,20,30]
5 print(10 not in c)     #False

```

运算符优先级问题

如下优先级，从高到低。

运算符	描述
**	指数 (最高优先级)
~	按位翻转
* / % //	乘, 除, 取模和取整除
+ -	加法减法
>> <<	右移, 左移运算符
&	位 'AND'
^	位运算符
<= < > >=	比较运算符
<> == !=	等于运算符
= %= /= //= -= += *= **=	赋值运算符
is is not	身份运算符
in not in	成员运算符
not or and	逻辑运算符

```

1  #测试运算符优先级
2  a,b,c,d=20,10,15,5
3  e=(a+b)*c/d          #30*15/5
4  print('(a+b)*c/d的执行结果:',e)
5  e=(a+b)*(c/d)        #30*(15/5)
6  print('(a+b)*(c/d)的执行结果:',e)
7  e=a+(b*c)/d          #20+150/5
8  print('a+(b*c)/d的执行结果:',e)

```

实际使用中, 记住如下简单的规则即可, 复杂的表达式一定要使用小括号组织。

- ① 乘除优先加减
- ② 位运算和算术运算>比较运算符>赋值运算符>逻辑运算符

基本运算符总结

运算符	说明
and, or, not	布尔与、布尔或、布尔非
is, is not	同一性判断, 判断是否为同一个对象
in, not in	判断是否在指定序列中
<, <=, >, >=, !=, ==	比较值是否相当, 可以连用
^ &	按位或, 按位异或、按位与
<<, >>	移位
~	按位翻转
+, -, *, /, //, %	加, 减, 乘, 浮点除、整数除、取余
**	幂运算

【操作】使用python表示数学式:

$$\frac{5+10x}{5} - \frac{13(y-1)(a+b)}{x} + 9\left(\frac{5}{x} + \frac{12+x}{y}\right)$$

答案: `(5+10*x)/5-13*(y-1)*(a+b)/x+9*(5/x+(12+x)/y)`

实时效果反馈

1. 如下代码, 错误的是:

- A** `10 in [10,20,30]` 结果是: `True`
- B** `"py" in "python"` 结果是: `True`
- C** `100 not in [10,20,30]` 结果是: `False`
- D** `"hello" is "hello"` 结果是: `True`

答案

1=>C

